

Infra as Code

TP #3 - Packer

Master 2
INFRA



l'école d'ingénierie
informatique

Introduction

Vous savez maintenant créer une infrastructure, vous savez la configurer, il nous reste plus qu'à toucher à la dernière brique élémentaire, le système. Actuellement, à chaque fois que vous instanciez un site Wordpress ou une base de donnée, vous devez tout re-provisionner à chaque fois. Normal puisqu'on démarre d'un OS vierge, mais coûteux puisque la procédure complète peut prendre environ 10-15 minutes. Nous allons donc voir comment accélérer ces délais en nous préparant des images prêtes à l'emploi.

L'objectif de ce projet étant de découvrir les rouages de packer, les seuls plugins autorisés sont donc *hashicorp/** 😊

Ce projet est à rendre sur Github (procédure identique à Terraform / Ansible). Si vous pouvez réfléchir ensemble, vous êtes invités à coder seuls car c'est tel que vous serez notés !

Contexte

Vous êtes toujours ingénieur DevOps chez « Best software Industries ». L'infra AWS est en place, l'offre « C-WordPress » fonctionne bien et est un carton planétaire. Cependant le nouveau VP est arrivé et la politique de l'entreprise change, la rentabilité est maintenant la clé ! Si 5 minutes peuvent être gagnées alors elles le doivent, l'avenir des dividendes en dépends !

Environnement de travail

Prérequis logiciels

Pour cette phase 2, vous aurez besoin :

1. De l'IDE de votre choix (VS Code, JetBrains ...)
2. De packer installé : [Install Packer](#)

Exécuter la commande suivante devrait vous donner quelque chose de similaire:

```
1  ?> packer version
2  Packer v1.15.
```

Prérequis externes

Github

Nous allons créer un nouveau repo pour ce projet. Veillez à n'y ajouter que le strict nécessaire.

En suivant la même procédure que pour le TP Terraform, invitez [@lombar_e](#) à ce repo.

AWS

Pour ce TP nous aurons besoins de droits supplémentaires sur AWS. A vous de les ajouter à votre utilisateur IAM.

Packer TP

Étape 0 - Vagrant

Actuellement, à chaque nouvel employé, nous devons re-provisionner et configurer chaque outil indépendamment. Afin de gagner du temps ici, votre RSSI vous demande de créer des Box Vagrant pré-configurés contenant tout ce qui est nécessaire aux développements avec Terraform, Ansible et AWS.

Votre box doit :

- Reposer sur *bento/rockylinux-9*
- La VM doit exposer les ports nécessaires si tel est le cas
- Le premier "*vagrant up*" doit éventuellement configurer ce qui est nécessaire
- Les outils suivants au moins doivent être installés dans notre box :
 - Aws cli
 - Terraform
 - Ansible
 - Packer
 - Docker

Notez que cet environnement vagrant doit vous permettre d'effectuer la suite du TP. Votre setup fonctionne si :

```
1  vagrant up # Doit provisionner sans intervention manuelle
2  vagrant ssh # Doit donner accès à la VM
3  terraform version # Doit répondre dans la VM
4  ansible --version # Doit répondre dans la VM
5  aws --version # Doit répondre dans la VM
```

Documentations de référence :

- [Vagrantfile documentation](#)
- [Provisioning](#)
- [Synced Folders](#)
- [Networking](#)

Étape 1 – ODOO en Local

Notre RSSI souhaite internaliser l'utilisation de nos CRMs. Il nous est donc nécessaire dans un premier temps de préparer l'encapsulation d'ODOO sur notre machine.

Cette étape consiste à préparer deux images via Packer. Une pour ODOO et une pour la base de donnée PostgreSQL. Les images docker de bases doivent être des Ubuntu.

Le livrable de cette étape repose sur :

- Un docker compose
 - Un service PostgreSQL
 - Un service ODOO
- L'ensemble doit être résilient aux redémarrages
- Bases de données, utilisateurs etc.. sont à configurer via un fichier d'environnement
- Un script *init.sql* qui pose le boilerplate de la base de donnée pour ODOO
- Odoo ne doit pas être configuré à chaque démarrage (volumes ?)
- Odoo ne doit démarrer qu'une fois que PostgreSQL est up et healthy

Documentations de référence :

- [Docker Compose Documentation](#)
- [Docker Hub Ubuntu](#)
- [ODOO installation documentation](#)
- [Postgres installation](#)

Étape 2 – ODOO en ligne !

Maintenant qu'on a validé nos configurations et installations, il nous reste plus qu'à rendre ça disponible en ligne.

Il faut pour ça déployer Odoo et PostgreSQL sur AWS, chaque service hébergé sur sa propre instance EC2 et provisionné par une AMI construite avec Packer.

AMIs à produire :

- Créez deux templates Packer dans un dossier packer/ :
 - packer/odoo.pkr.hcl :
 - AMI Amazon Linux 2023 contenant :
 - Odoo 17 Community installé et configuré
 - Le service Odoo activé mais non démarré
 - La connexion à PostgreSQL non définie dans l'image elle sera injectée au lancement de l'instance.
 - packer/postgresql.pkr.hcl :
 - AMI Amazon Linux 2023 contenant :
 - PostgreSQL 16 installé et configuré
 - L'utilisateur et la base de données Odoo créés
 - Le service PostgreSQL activé mais non démarré
- Les deux AMIs doivent être taguées de façon à être trouvables depuis Terraform.

Notez que vos deux templates doivent pouvoir produire une image docker ET une AMI.

Documentations de référence :

- [Packer builder EBS](#)

Etape 3 – Déployez ODOO sur AWS

Inspirez vous du projet Terraform du TP1 et provisionnez :

- Une instance **t3.micro** pour Odoo dans le sous-réseau public, construite depuis l'AMI Packer Odoo
- Une instance **t3.micro** pour PostgreSQL dans le sous-réseau privé, construite depuis l'AMI Packer PostgreSQL
- Les security groups adaptés :
 - Odoo accessible sur 8069 depuis internet
 - PostgreSQL accessible sur 5432 depuis l'instance Odoo uniquement
- Un script shell passé en **user_data** sur chaque instance qui démarre les services et injecte les configurations manquantes

Documentations de référence :

- [AWS Instance user_data](#)
- [data_source AMI](#)

Félicitations vous êtes arrivés au bout de ces trois jours !